



Université Euromed de Fès

École d'Ingénierie Digitale et Intelligence Artificielle

Rapport du TP : *Data Warehouse Moderne*

SCD Type 2 - Data Vault 2.0 - Architecture Lakehouse avec
PySpark & Delta Lake

Préparé par : Nada Ezzouhour El Achabi

Table des matières

1	Introduction	4
2	Source de données : PostgreSQL (OLTP)	5
2.1	Rôle de la base PostgreSQL dans le projet	5
2.2	Création de la base de données	5
2.2.1	Objectif de la création de la base	5
2.2.2	Instruction SQL utilisée	5
2.3	Création des tables sources	5
2.3.1	Table <code>clients_source</code>	5
2.3.2	Table <code>produits_source</code>	6
2.3.3	Table <code>ventes_source</code>	6
2.4	Insertion des données de test	7
2.4.1	Objectif des données de test	7
2.4.2	Intérêt pour le pipeline	7
2.5	Positionnement technique de PostgreSQL dans le pipeline	7
2.6	Conclusion de la couche OLTP	7
3	Gestion de l'historique des clients : SCD Type 2	8
3.1	Objectif du traitement SCD Type 2	8
3.2	Création de la table <code>dim_client</code>	8
3.2.1	Structure générale de la table	8
3.2.2	Code SQL de création	8
3.2.3	Description détaillée des colonnes	9
3.3	Script <code>03_load_scd_type2.py</code>	9
3.3.1	Rôle général du script	9
3.3.2	Chargement initial des clients	9
3.3.3	Détection des changements	9
3.3.4	Clôture de la version courante	10
3.3.5	Insertion de la nouvelle version	10
3.4	Résultat du traitement SCD Type 2	10
3.5	Utilisation de la dimension client dans le pipeline	10
3.6	Conclusion sur la gestion SCD Type 2	10
4	Modélisation Data Vault	11
4.1	Objectif de la modélisation Data Vault	11
4.2	Script <code>04_create_data_vault.sql</code>	11
4.2.1	Création des Hubs	11
4.2.2	Création du Link	12
4.2.3	Création des Satellites	12
4.3	Apports de la modélisation Data Vault dans le projet	13
5	Architecture Lakehouse avec Delta Lake	14
5.1	Principe général de l'architecture Lakehouse	14
5.2	Rôle de Delta Lake dans l'architecture	14
5.3	Organisation en couches du Lakehouse	14
5.4	Couche Bronze : ingestion des données brutes	15

5.5	Couche Silver : nettoyage et standardisation des données	15
5.6	Couche Gold : données analytiques et décisionnelles	15
5.7	Apports de l'architecture Lakehouse dans le projet	15
5.8	Conclusion sur l'architecture Lakehouse	16
6	Couche Bronze : Ingestion des données brutes	17
6.1	Rôle stratégique de la couche Bronze dans le pipeline	17
6.2	Présentation générale du script 05_bronze_ingestion.py	17
6.3	Initialisation de la session Spark	17
6.4	Connexion à la base PostgreSQL	18
6.5	Lecture des tables sources	18
6.6	Ajout de la colonne ingestion_timestamp	18
6.7	Ajout des métadonnées de provenance	18
6.8	Écriture des données au format Delta Lake	18
6.9	Absence volontaire de transformation métier	19
6.10	Importance de la couche Bronze pour les couches supérieures	19
7	Vérification approfondie de la couche Bronze	20
7.1	Enjeu et nécessité de la vérification de la couche Bronze	20
7.2	Objectif fonctionnel du script 06_verify_bronze.py	20
7.3	Initialisation contrôlée de la session Spark	20
7.4	Lecture explicite des tables Bronze au format Delta	21
7.5	Inspection du contenu des données	21
7.6	Analyse détaillée du schéma des données	21
7.7	Contrôle de la volumétrie des données	21
7.8	Validation finale de la couche Bronze	22
7.9	Apport de la phase de vérification à la qualité du pipeline	22
8	Couche Silver : Nettoyage et fiabilisation des données	23
8.1	Objectif de la couche Silver	23
8.2	Script 07_silver_transformation.py : Traitement des clients	23
8.2.1	Rôle général du script	23
8.2.2	Suppression des doublons	23
8.2.3	Normalisation des noms et prénoms	24
8.2.4	Validation du format des adresses email	24
8.3	Script 07b_silver_ventes_transformation.py : Traitement des ventes	24
8.3.1	Rôle général du script	24
8.3.2	Suppression des doublons de ventes	24
8.3.3	Filtrage des valeurs incohérentes	24
8.4	Apports de la couche Silver au pipeline analytique	25
8.5	Conclusion sur la couche Silver	25
9	Couche Gold : Données décisionnelles	26
9.1	Rôle et finalité de la couche Gold	26
9.2	Objectif analytique des indicateurs produits	26
9.3	Présentation du script 08_gold_aggregation.py	26
9.4	Lecture des données depuis la couche Silver	26
9.5	Transformation temporelle des données	27
9.6	Regroupement des ventes par jour	27
9.7	Calcul des agrégats analytiques	27
9.8	Tri et présentation des résultats	27
9.9	Écriture des données dans la couche Gold	27
9.10	Optimisation pour le reporting et la Business Intelligence	28
9.11	Conclusion sur la couche Gold	28
10	Génération du rapport final	29
10.0.1	Rôle du script 09_generer_rapport.py	29
10.0.2	Lecture des données issues de la couche Gold	29
10.0.3	Calcul des statistiques globales	29

10.0.4	Analyse des jours les plus performants	29
10.0.5	Production d'un rapport lisible et orienté métier	30
10.0.6	Finalité du pipeline de données	30
11	Conclusion	31
12	Captures d'écran	32

1 Introduction

Avec l’essor du numérique et la généralisation des systèmes d’information, les entreprises génèrent quotidiennement des volumes considérables de données issues de sources multiples telles que les bases transactionnelles, les applications métiers, les plateformes numériques ou encore les systèmes de gestion internes. Ces données constituent aujourd’hui un levier stratégique majeur pour les organisations, à condition qu’elles soient correctement collectées, stockées, structurées et analysées.

Dans la majorité des entreprises, les données opérationnelles sont gérées par des systèmes transactionnels, appelés **OLTP** (*Online Transaction Processing*). Ces systèmes sont principalement conçus pour assurer le bon déroulement des activités quotidiennes, telles que l’enregistrement des ventes, la gestion des clients ou la mise à jour des produits. Ils privilégient la rapidité des transactions, la cohérence des données et la gestion de la concurrence. Toutefois, leur architecture n’est pas adaptée aux besoins analytiques complexes, notamment lorsqu’il s’agit d’effectuer des analyses historiques, des agrégations massives ou des calculs décisionnels.

L’utilisation directe des bases OLTP à des fins analytiques pose plusieurs problématiques majeures. D’une part, les mises à jour fréquentes entraînent la perte de l’historique des données, rendant difficile toute analyse temporelle fiable. D’autre part, les requêtes analytiques lourdes peuvent dégrader significativement les performances des systèmes opérationnels, impactant ainsi les applications métiers. Enfin, la structure fortement normalisée des bases transactionnelles complique l’exploitation des données par les analystes et les outils de Business Intelligence.

Face à ces limites, les architectures de **Data Warehouse** se sont imposées comme une solution permettant de séparer les systèmes opérationnels des systèmes analytiques. Plus récemment, le concept de **Lakehouse** est apparu afin de combiner les avantages des Data Lakes (flexibilité, stockage massif) et des Data Warehouses (fiabilité, gouvernance, performance analytique). Ces architectures modernes reposent sur des pipelines de données capables d’ingérer, transformer, historiser et agréger les données de manière progressive et contrôlée.

Dans ce contexte, l’objectif de ce projet est de concevoir et de mettre en œuvre un **pipeline complet de Data Engineering**, depuis une base transactionnelle PostgreSQL jusqu’à la production d’indicateurs décisionnels exploitables. Le projet s’appuie sur des technologies modernes largement utilisées dans l’industrie, notamment **PySpark** pour le traitement distribué des données et **Delta Lake** pour le stockage transactionnel et fiable des données analytiques.

Ce travail met en application plusieurs concepts fondamentaux du Data Warehouse et du Big Data, parmi lesquels :

- la séparation claire entre systèmes **OLTP** et **OLAP**, afin de préserver les performances des applications opérationnelles ;
- la gestion de l’historique des données à l’aide du mécanisme des **Slowly Changing Dimensions de type 2 (SCD Type 2)**, permettant de conserver l’évolution des attributs dans le temps ;
- la modélisation **Data Vault**, reconnue pour sa capacité à assurer la traçabilité, l’historisation et l’évolutivité des données ;
- l’architecture **Bronze / Silver / Gold**, qui organise les données selon leur niveau de transformation et leur usage analytique.

Au-delà de l’aspect technique, ce projet vise à reproduire une architecture réaliste, proche des pratiques professionnelles actuelles en Data Engineering. Il illustre la manière dont des données brutes issues d’un système transactionnel peuvent être progressivement transformées en informations fiables et structurées, destinées à soutenir la prise de décision au sein d’une organisation.

2 Source de données : PostgreSQL (OLTP)

2.1 Rôle de la base PostgreSQL dans le projet

Dans ce projet, PostgreSQL joue le rôle de **système transactionnel (OLTP)**, c'est-à-dire la base de données utilisée pour stocker les informations opérationnelles générées par l'activité quotidienne de l'entreprise. Cette base constitue le **point d'entrée du pipeline de données** et représente la source officielle à partir de laquelle toutes les données analytiques sont extraites.

PostgreSQL a été choisi pour sa robustesse, sa conformité aux standards SQL et sa capacité à gérer efficacement des transactions concurrentes tout en garantissant l'intégrité des données. Il simule un environnement réel dans lequel les données sont continuellement insérées et mises à jour par les applications métiers.

2.2 Création de la base de données

2.2.1 Objectif de la création de la base

La première étape consiste à créer une base de données dédiée au projet, nommée `retailpro_dwh`. Cette séparation permet d'isoler les données du projet des autres bases existantes et de reproduire une organisation professionnelle des environnements de données.

2.2.2 Instruction SQL utilisée

La base est créée à l'aide de l'instruction suivante :

```
CREATE DATABASE retailpro_dwh;
```

Cette commande initialise un nouvel espace de stockage logique dans PostgreSQL, destiné à contenir les tables transactionnelles simulant le système opérationnel.

2.3 Création des tables sources

Les tables créées dans PostgreSQL représentent les entités métiers fondamentales du système : clients, produits et ventes. Elles sont conçues selon une logique OLTP classique, favorisant la normalisation et la cohérence des données.

2.3.1 Table `clients_source`

Rôle de la table

La table `clients_source` stocke les informations relatives aux clients. Elle représente une entité centrale du système, car les clients sont susceptibles d'évoluer dans le temps (changement de ville, de segment, etc.), ce qui justifie par la suite l'utilisation du SCD Type 2.

Structure et justification des colonnes

- **client_id** : clé primaire auto-incrémentée, utilisée comme identifiant unique du client dans le système opérationnel.
- **nom, prenom** : attributs descriptifs permettant d'identifier le client.
- **email** : champ unique garantissant l'unicité du client et évitant les doublons.
- **ville** : attribut susceptible d'évoluer dans le temps.
- **segment** : classification commerciale du client (Bronze, Silver, Gold, Platinum).
- **created_at** : date de création de l'enregistrement.
- **updated_at** : date de dernière modification, utilisée pour détecter les changements.

La présence des champs `created_at` et `updated_at` est essentielle pour suivre l'évolution des données et alimenter les mécanismes d'historisation en aval.

2.3.2 Table `produits_source`

Rôle de la table

La table `produits_source` contient les informations descriptives des produits proposés par l'entreprise. Contrairement aux clients, les produits évoluent généralement moins fréquemment, mais restent essentiels pour l'analyse des ventes et du chiffre d'affaires.

Structure et justification

- **produit_id** : clé primaire du produit.
- **nom_produit** : désignation du produit.
- **categorie** : catégorie marketing du produit.
- **prix_unitaire** : prix de vente.
- **cout_achat** : coût d'acquisition du produit.
- **created_at, updated_at** : métadonnées temporelles.

Ces attributs permettent de réaliser ultérieurement des analyses de rentabilité et de performance commerciale.

2.3.3 Table `ventes_source`

Rôle de la table

La table `ventes_source` représente les transactions de vente. Elle constitue le lien entre les clients et les produits et enregistre les événements générateurs de chiffre d'affaires.

Structure et justification

- **vente_id** : clé primaire de la transaction.
- **client_id** : clé étrangère vers `clients_source`.
- **produit_id** : clé étrangère vers `produits_source`.
- **date_vente** : date et heure de la transaction.
- **quantite** : nombre d'unités vendues.
- **montant_total** : montant total de la vente.

Les clés étrangères assurent l'intégrité référentielle entre les tables et reflètent les relations métiers réelles.

2.4 Insertion des données de test

2.4.1 Objectif des données de test

Des données fictives sont insérées afin de simuler une activité réelle et de permettre le test du pipeline complet. Ces données servent de base aux traitements ultérieurs dans les couches Bronze, Silver et Gold.

2.4.2 Intérêt pour le pipeline

Les données insérées permettent :

- de tester l’ingestion depuis PostgreSQL,
- de valider la détection des changements (SCD Type 2),
- de produire des agrégations analytiques cohérentes.

—

2.5 Positionnement technique de PostgreSQL dans le pipeline

PostgreSQL constitue la **source de vérité opérationnelle**. Les données ne sont jamais modifiées dans cette base à des fins analytiques. Elles sont extraites en lecture seule à l’aide de PySpark via un connecteur JDBC.

Cette approche garantit :

- la préservation des performances du système transactionnel,
- l’intégrité des données sources,
- une séparation claire entre les usages opérationnels et analytiques.

—

2.6 Conclusion de la couche OLTP

La base PostgreSQL mise en place dans ce projet constitue une représentation fidèle d’un système transactionnel réel. Elle fournit des données cohérentes et structurées, mais volontairement non adaptées à l’analyse décisionnelle directe. Cette contrainte justifie la mise en place du pipeline analytique, qui permettra de transformer progressivement ces données opérationnelles en informations exploitables.

3 Gestion de l'historique des clients : SCD Type 2

3.1 Objectif du traitement SCD Type 2

Le traitement SCD Type 2 a pour objectif de gérer l'évolution des informations clients tout en conservant l'intégralité de leur historique. Dans un système transactionnel classique, les mises à jour écrasent les anciennes valeurs, ce qui ne permet de conserver que l'état courant des données. Or, dans un contexte analytique, cette approche est insuffisante, car elle empêche toute analyse temporelle fiable.

Le mécanisme SCD Type 2 repose sur un principe simple : dès qu'un changement est détecté sur un ou plusieurs attributs descriptifs d'un client, une nouvelle version de ce client est créée dans la table de dimension. L'ancienne version n'est pas supprimée, mais clôturée, ce qui permet de conserver une trace complète des évolutions successives. Ainsi, chaque client peut être représenté par plusieurs lignes dans la dimension, chacune correspondant à une période de validité précise.

Dans le cadre de ce projet, le SCD Type 2 permet de relier correctement les faits (ventes) à l'état exact du client au moment où l'événement s'est produit. Il constitue donc un élément fondamental pour garantir la cohérence et la fiabilité des analyses décisionnelles réalisées en aval.

3.2 Création de la table `dim_client`

3.2.1 Structure générale de la table

La table `dim_client` a été conçue spécifiquement pour implémenter le mécanisme du SCD Type 2. Sa structure permet de stocker plusieurs versions d'un même client, tout en identifiant clairement la version courante et les versions historiques. Chaque ligne représente un état du client à une période donnée.

La conception de cette table repose sur l'utilisation d'une clé surrogate, distincte de la clé métier, ainsi que sur l'ajout de colonnes temporelles dédiées à la gestion de l'historique. Cette approche est conforme aux bonnes pratiques de modélisation en Data Warehouse.

3.2.2 Code SQL de création

```
CREATE TABLE dim_client (  
    client_key SERIAL PRIMARY KEY,  
    client_id INTEGER NOT NULL,  
    nom VARCHAR(100),  
    prenom VARCHAR(100),  
    email VARCHAR(200),  
    ville VARCHAR(100),  
    segment VARCHAR(50),  
    date_debut DATE NOT NULL DEFAULT CURRENT_DATE,  
    date_fin DATE,
```

```
est_courant BOOLEAN DEFAULT TRUE,  
version INTEGER DEFAULT 1,  
date_chargement TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
CONSTRAINT unique_client_version UNIQUE (client_id, version)  
);  
—
```

3.2.3 Description détaillée des colonnes

La colonne `client_key` est une clé surrogate générée automatiquement. Elle permet d'identifier de manière unique chaque version d'un client, indépendamment de son identifiant métier. Cette clé est utilisée dans les jointures avec les tables de faits afin d'assurer la stabilité des relations, même lorsque les informations du client évoluent.

La colonne `client_id` correspond à la clé métier issue de la table `clients_source` de PostgreSQL. Contrairement à la clé surrogate, elle peut apparaître plusieurs fois dans la table, chaque occurrence représentant une version différente du même client.

Les colonnes `nom`, `prenom`, `email`, `ville` et `segment` regroupent les attributs descriptifs du client. Ce sont ces champs qui sont surveillés lors du processus de détection des changements. Toute modification sur l'un de ces attributs entraîne la création d'une nouvelle version.

Les colonnes `date_debut` et `date_fin` définissent la période de validité de chaque version. Une valeur `NULL` dans `date_fin` indique que la version est toujours active. La colonne `est_courant` permet d'identifier rapidement la version courante sans avoir à analyser les dates.

Enfin, la colonne `version` stocke le numéro de version du client. Elle permet de suivre l'ordre chronologique des modifications et facilite la lecture et l'audit des données.

—

3.3 Script 03_load_scd_type2.py

3.3.1 Rôle général du script

Le script `03_load_scd_type2.py` assure l'automatisation complète du traitement SCD Type 2. Il constitue le lien entre la table source `clients_source` et la dimension `dim_client`. À chaque exécution, le script analyse les données sources et détermine si un client doit être inséré pour la première fois, mis à jour via une nouvelle version ou ignoré si aucune modification n'est détectée.

Ce script est conçu pour être relancé régulièrement. Il garantit ainsi que la dimension client reste synchronisée avec la source tout en conservant l'historique des évolutions.

—

3.3.2 Chargement initial des clients

Lors du premier lancement du script, la table `dim_client` est vide. Le script procède alors à l'insertion de l'ensemble des clients présents dans la table source. Chaque client est inséré avec une version initiale égale à 1, une date de début correspondant à la date de chargement, une date de fin à `NULL` et l'indicateur `est_courant` positionné à `TRUE`.

Cette étape permet d'établir un état de référence de la dimension client, à partir duquel toutes les évolutions futures seront enregistrées.

—

3.3.3 Détection des changements

Lors des exécutions suivantes, le script compare les données de la table `clients_source` avec la version courante de chaque client dans `dim_client`. Les comparaisons portent sur l'ensemble des attributs descriptifs susceptibles d'évoluer, notamment la ville, le segment et les informations de contact.

Si les valeurs sources sont identiques à celles de la version courante, aucune action n'est entreprise. En revanche, dès qu'une différence est détectée, le script déclenche le processus de création d'une nouvelle version.

3.3.4 Clôture de la version courante

Lorsqu'un changement est identifié, la version courante du client est clôturée. Cette opération consiste à renseigner la colonne `date_fin` avec la date du changement et à positionner l'indicateur `est_courant` à `FALSE`. Cette étape permet de conserver la version précédente tout en indiquant qu'elle n'est plus valide.

La clôture des versions garantit que, pour un client donné, une seule version est active à un instant donné.

3.3.5 Insertion de la nouvelle version

Après la clôture de l'ancienne version, une nouvelle ligne est insérée dans la table `dim_client`. Cette nouvelle version reprend les valeurs mises à jour provenant de la source, un numéro de version incrémenté, une date de début correspondant à la date du changement et une date de fin laissée à `NULL`. L'indicateur `est_courant` est positionné à `TRUE`.

Cette nouvelle ligne devient alors la version de référence du client pour les traitements analytiques futurs.

3.4 Résultat du traitement SCD Type 2

À l'issue du traitement, la table `dim_client` contient potentiellement plusieurs lignes pour un même client. Chaque ligne correspond à une période de validité distincte et permet de retracer précisément l'évolution des informations clients dans le temps.

Cette organisation rend possible l'analyse historique des données et garantit que les faits seront toujours associés à la version correcte du client.

3.5 Utilisation de la dimension client dans le pipeline

La dimension `dim_client` est utilisée comme référence dans les analyses décisionnelles. Elle permet de relier les ventes aux clients en tenant compte de leur état exact à la date de la transaction. Elle constitue ainsi un socle fiable pour les couches analytiques du Data Warehouse.

3.6 Conclusion sur la gestion SCD Type 2

La mise en œuvre du SCD Type 2 dans ce projet permet de transformer des données clients issues d'un système transactionnel en une dimension historisée et exploitable analytiquement. Cette approche garantit la traçabilité des évolutions, la cohérence des analyses temporelles et la fiabilité globale du pipeline de données.

4 Modélisation Data Vault

4.1 Objectif de la modélisation Data Vault

Dans ce projet, la modélisation Data Vault a été mise en œuvre afin de structurer les données analytiques de manière robuste et évolutive. Contrairement aux modèles classiques en étoile ou en flocon, le Data Vault est conçu pour absorber facilement les changements provenant des systèmes sources, tout en garantissant une historisation complète et une traçabilité précise des données.

L'objectif principal de l'utilisation du Data Vault est de séparer clairement les différents types d'informations manipulées dans le système. D'une part, il permet d'isoler les **identités métier** (telles que les clients ou les produits), qui sont généralement stables dans le temps. D'autre part, il distingue les **relations métier** entre ces identités, ainsi que les **attributs descriptifs**, qui sont susceptibles d'évoluer.

Cette séparation offre plusieurs avantages concrets dans le cadre du projet. Elle permet d'intégrer de nouvelles sources de données sans remettre en cause la structure existante, de conserver l'historique complet des modifications et de garantir une excellente traçabilité, depuis la donnée source jusqu'aux couches analytiques supérieures. Ainsi, la modélisation Data Vault constitue une base solide pour le Data Warehouse et le Lakehouse mis en place dans les étapes suivantes.

4.2 Script 04_create_data_vault.sql

Le script 04_create_data_vault.sql a pour rôle de créer l'ensemble des tables constituant le modèle Data Vault. Il définit explicitement les différentes composantes de ce modèle, à savoir les **Hubs**, les **Links** et les **Satellites**. Chacune de ces composantes remplit un rôle précis dans l'architecture globale.

4.2.1 Création des Hubs

Les Hubs représentent les entités métiers principales du système. Ils stockent les clés métiers uniques et stables, indépendamment des attributs descriptifs. Dans ce projet, deux Hubs ont été créés : hub_client et hub_produit.

Hub Client

La table hub_client contient les informations permettant d'identifier de manière unique un client. Elle repose sur une clé de hachage (client_hash_key) calculée à partir de la clé métier client_id. Cette clé de hachage permet de standardiser l'identification des entités et facilite les jointures au sein du Data Vault.

Le Hub Client ne contient aucun attribut descriptif tel que le nom ou la ville. Il se limite volontairement aux informations strictement nécessaires à l'identification du client, ainsi qu'aux métadonnées comme la date de chargement et la source des données. Cette conception garantit que le Hub reste stable, même si les informations descriptives du client évoluent dans le temps.

Hub Produit

La table `hub_produit` joue un rôle analogue pour les produits. Elle stocke la clé métier `produit_id`, associée à une clé de hachage `produit_hash_key`. Comme pour le Hub Client, cette table ne contient que des informations d'identification et des métadonnées.

Cette approche permet de représenter de manière cohérente les produits dans le Data Vault, indépendamment des changements futurs sur leurs attributs descriptifs (prix, catégorie, etc.).

4.2.2 Création du Link

Le Link représente les relations métier entre les entités identifiées dans les Hubs. Dans ce projet, la table `link_vente` modélise la relation entre les clients et les produits à travers les transactions de vente.

La table `link_vente` contient une clé de hachage `vente_hash_key`, construite à partir des identifiants de la vente, du client et du produit. Elle référence explicitement les clés de hachage des Hubs Client et Produit, ce qui permet de relier les entités sans dupliquer leurs informations.

Cette table joue un rôle central dans le modèle, car elle matérialise les interactions métier réelles. Elle permet de représenter les ventes comme des relations indépendantes des attributs descriptifs, assurant ainsi une grande flexibilité en cas d'évolution du modèle ou des sources de données.

4.2.3 Création des Satellites

Les Satellites contiennent les attributs descriptifs et leur historique. Contrairement aux Hubs et aux Links, ils sont conçus pour évoluer dans le temps. Dans ce projet, trois Satellites ont été créés : `sat_client`, `sat_produit` et `sat_vente`.

Satellite Client

La table `sat_client` stocke les informations descriptives du client, telles que le nom, le prénom, l'email, la ville et le segment. Elle est liée au Hub Client via la clé de hachage `client_hash_key`.

Chaque enregistrement du Satellite Client correspond à une version des informations du client à une date donnée. La présence des colonnes `load_date` et `load_end_date` permet de gérer l'historique des modifications, tandis que la colonne `hash_diff` facilite la détection des changements entre deux versions successives.

Satellite Produit

La table `sat_produit` contient les attributs descriptifs des produits, tels que le nom, la catégorie, le prix unitaire et le coût d'achat. Comme pour le Satellite Client, chaque modification entraîne l'insertion d'une nouvelle version, garantissant ainsi la conservation de l'historique.

Cette table permet notamment d'analyser l'évolution des prix ou des catégories produits dans le temps, sans impacter la structure des Hubs ou des Links.

Satellite Vente

Le Satellite Vente stocke les attributs descriptifs liés aux transactions, tels que la date de vente, la quantité et le montant total. Il est rattaché au Link Vente via la clé de hachage `vente_hash_key`.

Cette séparation permet de conserver un historique détaillé des ventes, tout en maintenant une structure claire et modulaire.

4.3 Apports de la modélisation Data Vault dans le projet

La mise en place du Data Vault dans ce projet garantit une stabilité structurelle élevée, même en cas d'évolution du système source. L'ajout de nouveaux attributs ou de nouvelles sources de données peut être réalisé en créant de nouveaux Satellites, sans modification des Hubs existants.

De plus, cette modélisation assure une traçabilité complète des données, depuis leur origine jusqu'aux couches analytiques supérieures. Elle constitue ainsi une base fiable et évolutive pour l'architecture Lakehouse et pour les analyses décisionnelles réalisées ultérieurement.

5 Architecture Lakehouse avec Delta Lake

5.1 Principe général de l'architecture Lakehouse

Dans ce projet, une architecture de type **Lakehouse** a été mise en place afin de combiner les avantages des architectures traditionnelles de Data Lake et de Data Warehouse. Le Data Lake permet un stockage flexible et massif des données, sans contrainte forte sur le schéma, tandis que le Data Warehouse offre des garanties en termes de qualité, de gouvernance et de performance analytique. Le Lakehouse vise à réunir ces deux approches au sein d'une architecture unifiée.

Concrètement, l'architecture Lakehouse permet de stocker les données dans des formats ouverts, tout en bénéficiant de mécanismes avancés tels que la gestion des transactions, le contrôle de la qualité des données, la gestion de l'historique et l'optimisation des performances analytiques. Dans ce projet, cette architecture repose sur l'utilisation de **Delta Lake**, une couche de stockage transactionnelle qui s'intègre nativement avec Apache Spark.

Le choix du Lakehouse répond à un objectif clair : disposer d'une architecture capable d'évoluer avec les volumes de données, tout en garantissant la fiabilité et la traçabilité nécessaires à un usage analytique et décisionnel.

5.2 Rôle de Delta Lake dans l'architecture

Delta Lake joue un rôle central dans l'implémentation du Lakehouse. Il s'agit d'une couche de stockage qui apporte des fonctionnalités avancées au-dessus d'un stockage de type Data Lake. Parmi ces fonctionnalités, on retrouve notamment la gestion des transactions ACID, le versioning des données, la possibilité de rejouer des traitements et la détection des incohérences.

Dans le cadre de ce projet, toutes les données stockées dans les couches Bronze, Silver et Gold sont enregistrées au format Delta. Ce choix garantit que chaque étape du pipeline repose sur des données fiables, cohérentes et traçables. En cas d'erreur lors d'un traitement, il est possible de revenir à une version précédente des données sans perte d'information, ce qui constitue un avantage majeur pour la maintenance et l'audit du pipeline.

5.3 Organisation en couches du Lakehouse

L'architecture Lakehouse mise en place dans ce projet est organisée en trois couches distinctes : **Bronze**, **Silver** et **Gold**. Cette organisation permet de structurer progressivement les données en fonction de leur niveau de transformation et de leur usage analytique.

Chaque couche a un rôle bien défini et répond à des objectifs précis. La séparation en couches facilite la compréhension du pipeline, améliore la maintenabilité du système et permet d'isoler les responsabilités à chaque étape du traitement.

5.4 Couche Bronze : ingestion des données brutes

La couche Bronze constitue le premier niveau du Lakehouse. Elle contient une copie fidèle des données issues des systèmes sources, en l'occurrence la base PostgreSQL. Les données y sont stockées sans transformation métier significative, afin de préserver leur structure et leur contenu d'origine.

Dans ce projet, les tables `clients_source`, `produits_source` et `ventes_source` sont extraites depuis PostgreSQL à l'aide de PySpark et écrites au format Delta dans la couche Bronze. Des métadonnées supplémentaires, telles que la date d'ingestion et le système source, sont ajoutées afin d'assurer la traçabilité des données.

L'objectif principal de la couche Bronze est de servir de **zone d'atterrissage** des données. Elle permet de rejouer les traitements en cas d'erreur et constitue une référence stable à partir de laquelle les transformations ultérieures peuvent être appliquées. Aucune logique métier complexe n'est introduite à ce niveau, ce qui garantit la fidélité des données par rapport à la source.

5.5 Couche Silver : nettoyage et standardisation des données

La couche Silver correspond à un niveau intermédiaire de transformation des données. Les données issues de la couche Bronze y sont nettoyées, filtrées et standardisées afin d'améliorer leur qualité et leur cohérence.

Dans ce projet, la couche Silver applique plusieurs règles de traitement, telles que la suppression des doublons, la normalisation des formats de texte (majuscules, capitalisation), la validation des adresses email et l'élimination des valeurs incohérentes. Ces opérations permettent de produire des données fiables, prêtes à être utilisées pour l'analyse.

La couche Silver joue un rôle fondamental dans le pipeline, car elle constitue le point de transition entre les données brutes et les données analytiques. Elle permet de corriger les imperfections des données sources sans altérer leur signification métier. Les données stockées à ce niveau sont déjà exploitables, mais restent suffisamment détaillées pour permettre des transformations plus avancées dans la couche suivante.

5.6 Couche Gold : données analytiques et décisionnelles

La couche Gold représente le niveau le plus élevé du Lakehouse. Elle contient des données agrégées et structurées spécifiquement pour répondre aux besoins analytiques et décisionnels. Les données y sont organisées sous forme d'indicateurs clés de performance (KPI), de statistiques ou de tables de synthèse.

Dans ce projet, la couche Gold est utilisée pour calculer des agrégations telles que le chiffre d'affaires quotidien, le nombre de ventes par jour et le panier moyen. Ces indicateurs sont obtenus à partir des données nettoyées de la couche Silver, puis stockés au format Delta pour garantir leur fiabilité et leur performance d'accès.

La couche Gold est destinée à être consommée directement par des outils de reporting, de Business Intelligence ou par des scripts de génération de rapports. Elle constitue la finalité du pipeline de données, car elle transforme des données brutes en informations directement exploitables pour la prise de décision.

5.7 Apports de l'architecture Lakehouse dans le projet

L'architecture Lakehouse mise en place dans ce projet apporte plusieurs bénéfices majeurs. Elle permet une séparation claire des responsabilités entre les différentes étapes du traitement, améliore la qualité globale des données et facilite la maintenance du pipeline. Grâce à l'utilisation de Delta Lake, elle garantit également la cohérence, la traçabilité et la robustesse des données à chaque niveau.

Cette architecture offre enfin une grande évolutivité. De nouvelles sources de données, de nouvelles règles de transformation ou de nouveaux indicateurs peuvent être intégrés sans remettre en cause l'existant. Le Lakehouse constitue ainsi une base solide et moderne pour le développement de solutions analytiques à grande échelle.

5.8 Conclusion sur l'architecture Lakehouse

En résumé, l'architecture Lakehouse adoptée dans ce projet permet de structurer efficacement le cycle de vie des données, depuis leur ingestion brute jusqu'à leur exploitation décisionnelle. En combinant les avantages du Data Lake et du Data Warehouse, elle offre une solution flexible, performante et adaptée aux besoins actuels du Data Engineering.

6 Couche Bronze : Ingestion des données brutes

6.1 Rôle stratégique de la couche Bronze dans le pipeline

La couche Bronze représente la première étape opérationnelle du pipeline de données mis en place dans ce projet. Elle constitue le point de transition entre le système transactionnel, en l'occurrence la base de données PostgreSQL, et l'architecture analytique Lakehouse. Son rôle est fondamental, car elle assure la capture et la conservation des données dans leur forme la plus proche possible de la source.

Contrairement aux couches analytiques supérieures, la couche Bronze ne vise pas à améliorer la qualité des données ni à répondre directement à des besoins décisionnels. Elle a pour mission principale de garantir l'exactitude, l'exhaustivité et la traçabilité des données ingérées. Toute donnée présente dans les couches Silver ou Gold doit pouvoir être retrouvée, sans ambiguïté, dans la couche Bronze.

Dans le cadre de ce projet, la couche Bronze joue également un rôle de sécurité. Elle permet de découpler complètement les traitements analytiques du système source, évitant ainsi toute sollicitation inutile de la base PostgreSQL. En cas d'erreur dans une transformation ultérieure, les données peuvent être retraitées à partir de la couche Bronze sans avoir à relancer une extraction depuis la source transactionnelle.

6.2 Présentation générale du script `05_bronze_ingestion.py`

Le script `05_bronze_ingestion.py` est responsable de l'ingestion des données depuis PostgreSQL vers la couche Bronze du Lakehouse. Il a été implémenté en PySpark afin de tirer parti des capacités de traitement distribuées et de l'intégration native avec Delta Lake.

Ce script suit une logique structurée et répétable. Il applique exactement le même processus à plusieurs tables sources, garantissant ainsi une homogénéité de traitement. Cette approche facilite la maintenance du pipeline et permet d'ajouter de nouvelles tables sources sans remettre en cause l'architecture existante.

6.3 Initialisation de la session Spark

La première étape du script consiste à initialiser une session Spark configurée pour fonctionner avec Delta Lake. Cette étape est indispensable, car elle permet à Spark de reconnaître le format Delta et d'activer les fonctionnalités avancées associées, telles que la gestion des transactions et le versioning des données.

La session Spark est configurée en mode local, ce qui est adapté au contexte du projet et à un environnement de développement ou de travaux pratiques. Le niveau de journalisation est volontairement réduit afin de limiter le volume des messages affichés lors de l'exécution du script et de se concentrer sur les informations essentielles.

Cette initialisation constitue le socle technique sur lequel reposent toutes les opérations d'ingestion réalisées par la suite.

6.4 Connexion à la base PostgreSQL

Une fois la session Spark créée, le script établit une connexion à la base PostgreSQL via le connecteur JDBC. Cette méthode permet de lire directement les tables relationnelles tout en conservant leur schéma et leurs types de données.

Les paramètres de connexion, tels que l'URL de la base, le nom de l'utilisateur, le mot de passe et le driver JDBC, sont explicitement définis afin d'assurer une connexion fiable et sécurisée. Cette approche garantit que les données sont extraites sans transformation préalable et sans impact sur le fonctionnement du système transactionnel.

Dans ce projet, cette connexion est utilisée pour accéder aux tables `clients_source`, `produits_source` et `ventes_source`, qui constituent les principales entités métier du système.

6.5 Lecture des tables sources

Chaque table source est lue séparément et chargée dans un DataFrame Spark. Cette étape permet de manipuler les données de manière structurée tout en conservant leur organisation relationnelle d'origine.

La lecture table par table offre plusieurs avantages. Elle permet de contrôler précisément le contenu de chaque DataFrame, de vérifier le nombre de lignes extraites et d'identifier rapidement d'éventuelles anomalies dans les données sources. Elle facilite également l'ajout ultérieur de règles spécifiques à certaines tables, si nécessaire.

À ce stade du traitement, aucune transformation n'est appliquée aux colonnes métiers. Les données sont strictement identiques à celles présentes dans PostgreSQL.

6.6 Ajout de la colonne `ingestion_timestamp`

Après la lecture des données, le script enrichit chaque DataFrame avec une colonne `ingestion_timestamp`. Cette colonne enregistre la date et l'heure exactes auxquelles les données ont été ingérées dans la couche Bronze.

Cette information est cruciale pour assurer la traçabilité temporelle des données. Elle permet de savoir précisément quand une version donnée des données a été extraite du système source. En cas de divergence entre la source et le Lakehouse, cette colonne facilite l'analyse des écarts et la compréhension de leur origine.

L'ajout de cette colonne ne modifie en aucun cas les données métiers, mais fournit un contexte indispensable à la gouvernance des données.

6.7 Ajout des métadonnées de provenance

En complément de la date d'ingestion, le script ajoute deux colonnes supplémentaires : `source_system` et `source_table`. Ces métadonnées permettent d'identifier clairement l'origine de chaque enregistrement stocké dans la couche Bronze.

La colonne `source_system` indique que les données proviennent de PostgreSQL, tandis que la colonne `source_table` précise le nom exact de la table source. Cette information est particulièrement utile dans des architectures intégrant plusieurs systèmes sources, car elle garantit une traçabilité complète et explicite.

Grâce à ces métadonnées, chaque ligne de la couche Bronze peut être reliée sans ambiguïté à sa source d'origine.

6.8 Écriture des données au format Delta Lake

Une fois les données enrichies par les métadonnées, elles sont écrites dans la couche Bronze au format Delta Lake. Chaque table est stockée dans un répertoire distinct, ce qui permet de maintenir une organisation claire et logique du Lakehouse.

Le choix du format Delta Lake apporte plusieurs garanties techniques. Il permet d'assurer la cohérence des données grâce aux transactions ACID, même en cas d'interruption du traitement. Il offre également la possibilité de conserver un historique des versions, facilitant ainsi le débogage et la reprise des traitements.

L'écriture est réalisée de manière contrôlée afin de garantir que la couche Bronze reste une représentation fidèle et fiable des données sources.

6.9 Absence volontaire de transformation métier

Un principe fondamental respecté dans ce script est l'absence totale de transformation métier. Aucune règle de nettoyage, de filtrage ou d'agrégation n'est appliquée aux données à ce stade du pipeline.

Ce choix est volontaire et méthodologique. Il permet de garantir que la couche Bronze reste une copie exacte des données sources, enrichie uniquement par des métadonnées techniques. Toute transformation susceptible de modifier la signification métier des données est repoussée vers la couche Silver.

Cette séparation stricte des responsabilités contribue à la clarté, à la maintenabilité et à la robustesse du pipeline global.

6.10 Importance de la couche Bronze pour les couches supérieures

La couche Bronze constitue la base sur laquelle reposent l'ensemble des transformations ultérieures. Elle permet de reconstruire les couches Silver et Gold à tout moment, sans dépendre à nouveau du système source.

Grâce à cette architecture, le pipeline devient plus résilient face aux erreurs, plus facile à faire évoluer et plus transparent du point de vue de la gouvernance des données. La couche Bronze joue ainsi un rôle central dans la fiabilité globale du système décisionnel mis en place dans ce projet.

7 Vérification approfondie de la couche Bronze

7.1 Enjeu et nécessité de la vérification de la couche Bronze

Dans un pipeline de données structuré, la phase de vérification constitue une étape critique qui permet d'assurer la fiabilité globale de l'architecture avant d'engager des transformations plus complexes. La couche Bronze, en tant que première couche du Lakehouse, sert de fondation à l'ensemble du pipeline. Toute erreur ou incohérence à ce niveau se propagerait mécaniquement vers les couches Silver et Gold, compromettant ainsi la qualité des analyses finales.

La vérification de la couche Bronze vise donc à confirmer que l'ingestion des données depuis le système source a été réalisée de manière complète, fidèle et techniquement correcte. Elle permet notamment de s'assurer que les données ont bien été stockées au format Delta Lake, que leur structure est conforme aux attentes et que les métadonnées d'ingestion ont été correctement ajoutées. Cette étape joue également un rôle préventif, en permettant de détecter rapidement des problèmes potentiels avant qu'ils n'impactent l'ensemble du pipeline.

7.2 Objectif fonctionnel du script `06_verify_bronze.py`

Le script `06_verify_bronze.py` a été conçu comme un outil de contrôle et de validation de la couche Bronze. Son objectif n'est pas de transformer ou de modifier les données, mais uniquement de les lire, de les inspecter et d'en vérifier la cohérence. Il s'inscrit dans une démarche de contrôle qualité, comparable à une phase de tests intermédiaires dans un processus de développement logiciel.

Ce script permet de valider plusieurs aspects essentiels de l'ingestion : l'accessibilité des tables Delta, la présence des données attendues, la conformité du schéma et la cohérence de la volumétrie. En centralisant ces vérifications dans un script dédié, il devient possible de répéter facilement les contrôles après chaque ingestion ou modification du pipeline.

7.3 Initialisation contrôlée de la session Spark

La première opération réalisée par le script consiste à initialiser une session Spark configurée pour la lecture de tables Delta Lake. Cette étape est indispensable, car Spark doit être explicitement configuré pour interpréter le format Delta et accéder aux métadonnées associées aux fichiers stockés.

La session Spark est créée en mode local, ce qui correspond au contexte d'exécution du projet. Le niveau de journalisation est volontairement réglé sur un niveau modéré afin de limiter l'affichage des messages techniques non essentiels. Cette configuration permet de se concentrer sur les résultats de la vérification tout en conservant la possibilité d'analyser les messages d'avertissement si nécessaire.

Cette étape garantit que l'environnement d'exécution utilisé pour la vérification est cohérent avec celui utilisé lors de l'ingestion, ce qui renforce la fiabilité des contrôles effectués.

7.4 Lecture explicite des tables Bronze au format Delta

Une fois la session Spark initialisée, le script procède à la lecture des tables stockées dans la couche Bronze à l'aide de l'instruction `spark.read.format("delta")`. Cette méthode permet de charger directement les données depuis le stockage Lakehouse sans passer par le système source.

Dans le cadre du projet, la table des clients est utilisée comme table de référence pour la vérification. Le chargement réussi de cette table confirme que les fichiers Delta sont accessibles, que le chemin de stockage est correct et que les métadonnées Delta n'ont pas été corrompues lors de l'ingestion.

Cette étape constitue une validation technique essentielle, car toute erreur à ce niveau indiquerait un problème dans l'écriture des données ou dans la configuration de l'environnement Spark.

7.5 Inspection du contenu des données

Après le chargement des données dans un `DataFrame` Spark, le script affiche un échantillon des enregistrements à l'aide de la méthode `show()`. Cette inspection visuelle permet de vérifier que les données métiers correspondent bien aux valeurs présentes dans la base PostgreSQL au moment de l'ingestion.

L'affichage permet notamment de confirmer que les identifiants, les attributs descriptifs et les valeurs associées n'ont subi aucune altération involontaire. Il permet également de vérifier la présence des colonnes techniques ajoutées lors de l'ingestion, telles que la date d'ingestion et les informations de provenance.

Cette étape, bien que simple en apparence, joue un rôle important dans la validation humaine du pipeline, en complément des contrôles purement techniques.

7.6 Analyse détaillée du schéma des données

Le script affiche ensuite le schéma détaillé du `DataFrame` à l'aide de la méthode `printSchema()`. Cette opération permet de vérifier que la structure des données est conforme aux attentes, tant en termes de noms de colonnes que de types de données.

La vérification du schéma est cruciale pour garantir la compatibilité des données avec les transformations prévues dans la couche Silver. Un type incorrect ou une colonne manquante pourrait entraîner des erreurs lors des traitements ultérieurs. En validant le schéma à ce stade, le projet réduit considérablement le risque d'erreurs en cascade dans le pipeline.

Cette analyse permet également de s'assurer que les colonnes de métadonnées ont été correctement ajoutées et que leur typage est cohérent avec leur usage prévu.

7.7 Contrôle de la volumétrie des données

En complément de l'inspection visuelle et structurelle, le script calcule le nombre total de lignes présentes dans la table Bronze à l'aide de la méthode `count()`. Cette information est comparée implicitement à la volumétrie connue de la table source dans PostgreSQL.

Le contrôle de la volumétrie permet de détecter des anomalies telles qu'une perte de données, une duplication involontaire ou une lecture partielle de la source. Dans le contexte du projet, l'égalité entre le nombre de lignes de la source et celui de la couche Bronze confirme que l'ingestion a été réalisée de manière exhaustive.

Ce contrôle constitue un indicateur quantitatif simple mais efficace de la réussite de l'ingestion.

7.8 Validation finale de la couche Bronze

L'ensemble des vérifications réalisées par le script `06_verify_bronze.py` permet de conclure sur la validité de la couche Bronze. La lecture sans erreur des tables Delta, la conformité des données affichées, la cohérence du schéma et la correspondance de la volumétrie constituent des preuves tangibles du bon déroulement de l'ingestion.

Cette validation autorise le passage aux étapes suivantes du pipeline, notamment les transformations de la couche Silver. Elle garantit que ces transformations reposeront sur une base de données fiable, traçable et conforme aux données sources.

7.9 Apport de la phase de vérification à la qualité du pipeline

La phase de vérification de la couche Bronze renforce significativement la qualité globale du pipeline de données. En introduisant un point de contrôle explicite entre l'ingestion et les transformations, elle améliore la robustesse, la maintenabilité et la fiabilité de l'architecture.

Cette approche méthodique s'inscrit dans une démarche professionnelle de Data Engineering, où chaque étape du pipeline est validée avant de poursuivre les traitements. Elle contribue ainsi à garantir la qualité des analyses et des rapports produits en fin de chaîne.

8 Couche Silver : Nettoyage et fiabilisation des données

8.1 Objectif de la couche Silver

La couche Silver représente une étape intermédiaire essentielle dans l'architecture Lakehouse mise en place dans ce projet. Son objectif principal est de transformer les données brutes issues de la couche Bronze en données propres, cohérentes et fiables, tout en conservant leur niveau de détail. Contrairement à la couche Bronze, qui se limite à stocker les données telles qu'elles proviennent du système source, la couche Silver introduit une logique de qualité des données.

Cette couche vise à corriger les imperfections inhérentes aux systèmes transactionnels, telles que les doublons, les incohérences de format ou les valeurs aberrantes. Ces anomalies, bien que tolérables dans un contexte opérationnel, peuvent fausser les analyses si elles ne sont pas traitées en amont. La couche Silver joue donc un rôle de filtre et de standardisation, garantissant que seules des données exploitables sont transmises aux couches analytiques supérieures.

Dans ce projet, la couche Silver constitue le socle sur lequel reposent les agrégations de la couche Gold. Elle garantit que les indicateurs calculés ultérieurement reposent sur des données fiables et comparables dans le temps.

8.2 Script `07_silver_transformation.py` : Traitement des clients

8.2.1 Rôle général du script

Le script `07_silver_transformation.py` est dédié au traitement des données clients issues de la couche Bronze. Il applique un ensemble de règles de nettoyage et de standardisation visant à améliorer la qualité des données sans modifier leur signification métier.

Ce script transforme les données clients brutes en un jeu de données homogène et cohérent, prêt à être utilisé dans des analyses ou des agrégations ultérieures. Il s'inscrit dans une logique de préparation des données, indispensable avant toute exploitation analytique.

8.2.2 Suppression des doublons

La première transformation appliquée dans ce script consiste à supprimer les doublons présents dans les données clients. Dans un système transactionnel, il est possible qu'un même client apparaisse plusieurs fois en raison de réinsertions, d'erreurs de saisie ou de synchronisations multiples.

La suppression des doublons est réalisée sur la base de l'identifiant client (`client_id`), qui constitue la clé métier de référence. Cette opération garantit qu'un client n'est représenté qu'une seule fois dans la couche Silver. Elle évite ainsi les biais analytiques, tels que la surreprésentation d'un client dans les calculs ou les agrégations futures.

Cette étape contribue directement à la fiabilité des analyses, car elle assure une correspondance univoque entre un client et ses attributs descriptifs.

8.2.3 Normalisation des noms et prénoms

La normalisation des champs textuels constitue une étape importante du nettoyage des données. Dans ce projet, les noms de famille sont convertis en majuscules, tandis que les prénoms sont capitalisés (première lettre en majuscule, les suivantes en minuscules).

Cette transformation permet d'éliminer les variations de format dues à des saisies hétérogènes dans le système source, telles que des différences de casse ou de style. En standardisant les valeurs textuelles, la couche Silver garantit une meilleure lisibilité des données et facilite les comparaisons, les regroupements et les analyses ultérieures.

Cette normalisation n'altère en rien la signification métier des données, mais améliore leur cohérence visuelle et structurelle.

8.2.4 Validation du format des adresses email

La validation des adresses email constitue une autre étape clé du nettoyage des données clients. Les adresses email incorrectes ou mal formées peuvent poser des problèmes lors de l'analyse ou de l'utilisation des données à des fins de communication ou de segmentation.

Dans le script, une expression régulière est utilisée pour vérifier que les adresses email respectent un format standard. Les enregistrements ne satisfaisant pas cette condition sont exclus de la couche Silver. Cette approche permet de garantir que seules des adresses email valides sont conservées pour les analyses futures.

Cette transformation améliore la qualité globale des données clients et renforce la fiabilité des analyses basées sur les informations de contact.

8.3 Script `07b_silver_ventes_transformation.py` : Traitement des ventes

8.3.1 Rôle général du script

Le script `07b_silver_ventes_transformation.py` est dédié au nettoyage des données de ventes issues de la couche Bronze. Il applique des règles de contrôle visant à éliminer les incohérences et à garantir la validité des transactions enregistrées.

Les données de ventes jouent un rôle central dans les analyses décisionnelles, notamment pour le calcul du chiffre d'affaires et des indicateurs de performance. Il est donc essentiel de s'assurer de leur qualité avant de procéder à des agrégations.

8.3.2 Suppression des doublons de ventes

Comme pour les données clients, la première transformation appliquée aux ventes consiste à supprimer les doublons. Une même transaction peut apparaître plusieurs fois en raison d'erreurs de chargement ou de synchronisation.

La suppression des doublons permet de garantir que chaque vente est comptabilisée une seule fois dans les analyses. Cette étape est essentielle pour éviter une surestimation du chiffre d'affaires ou du nombre de ventes.

8.3.3 Filtrage des valeurs incohérentes

Le script applique également des règles de filtrage sur les valeurs numériques, en particulier la quantité vendue et le montant total de la vente. Les enregistrements présentant des valeurs négatives ou nulles sont considérés comme incohérents et sont exclus de la couche Silver.

Ce filtrage permet d'éliminer des transactions manifestement erronées, qui pourraient fausser les analyses financières. Il garantit que seules des ventes valides et économiquement cohérentes sont prises en compte dans les calculs analytiques.

8.4 Apports de la couche Silver au pipeline analytique

La couche Silver joue un rôle déterminant dans la qualité globale du pipeline de données. En appliquant des règles de nettoyage et de standardisation, elle transforme des données brutes en données fiables, prêtes à être agrégées et analysées.

Grâce à cette couche, les analyses réalisées dans la couche Gold reposent sur des données cohérentes et comparables, ce qui renforce la crédibilité des indicateurs produits. La couche Silver constitue ainsi un maillon essentiel entre l'ingestion brute des données et leur exploitation décisionnelle.

8.5 Conclusion sur la couche Silver

En conclusion, la couche Silver permet d'assurer un haut niveau de qualité des données avant leur utilisation analytique. Elle corrige les anomalies les plus courantes issues des systèmes transactionnels et garantit que les données transmises aux couches supérieures sont propres, cohérentes et exploitables. Cette étape est indispensable pour garantir la fiabilité des résultats produits par le Data Warehouse.

9 Couche Gold : Données décisionnelles

9.1 Rôle et finalité de la couche Gold

La couche Gold constitue l'aboutissement du pipeline de données mis en place dans ce projet. Elle représente le niveau le plus élevé de transformation et a pour objectif de fournir des données directement exploitables à des fins décisionnelles. Contrairement aux couches Bronze et Silver, qui conservent un niveau de détail important, la couche Gold est spécifiquement conçue pour répondre à des besoins analytiques précis.

Dans cette couche, les données ne sont plus destinées à être explorées ou nettoyées, mais à être interprétées. Elles prennent la forme d'indicateurs métiers synthétiques, tels que le chiffre d'affaires, le nombre de ventes ou le panier moyen. Ces indicateurs permettent aux décideurs d'avoir une vision claire et immédiate de la performance de l'activité, sans avoir à manipuler des données transactionnelles brutes.

La couche Gold est ainsi optimisée pour être consommée par des outils de reporting, de Business Intelligence ou par des scripts de génération de rapports. Elle joue un rôle central dans la transformation des données en information à forte valeur ajoutée.

9.2 Objectif analytique des indicateurs produits

Les indicateurs calculés dans la couche Gold ont été choisis pour répondre à des besoins analytiques concrets. Le nombre de ventes permet de mesurer l'activité commerciale sur une période donnée. Le chiffre d'affaires constitue un indicateur financier clé, reflétant la performance économique globale. Le panier moyen permet quant à lui d'analyser le comportement d'achat des clients et d'évaluer la valeur moyenne d'une transaction.

Ces indicateurs sont calculés à partir des données nettoyées de la couche Silver, ce qui garantit leur fiabilité. Ils peuvent être analysés dans le temps, comparés entre différentes périodes ou utilisés comme base pour des tableaux de bord décisionnels.

9.3 Présentation du script `08_gold_aggregation.py`

Le script `08_gold_aggregation.py` est chargé de la création de la couche Gold. Il s'appuie sur PySpark afin de traiter efficacement les données issues de la couche Silver et de calculer les agrégations nécessaires.

Ce script est volontairement centré sur des opérations analytiques. Contrairement aux scripts précédents, il n'introduit aucune logique de nettoyage ou de correction des données. Il suppose que les données en entrée sont déjà fiables et cohérentes, ce qui est garanti par les traitements réalisés dans la couche Silver.

9.4 Lecture des données depuis la couche Silver

La première étape du script consiste à lire les données de ventes stockées dans la couche Silver. Ces données ont déjà été nettoyées et validées, ce qui permet de les utiliser directement pour les calculs analytiques.

La lecture est effectuée au format Delta Lake, ce qui garantit la cohérence des données et permet de bénéficier des mécanismes de gestion des transactions. Cette étape marque le passage définitif du niveau opérationnel au niveau analytique.

9.5 Transformation temporelle des données

Avant de procéder aux agrégations, le script transforme la colonne `date_vente`, initialement de type `timestamp`, en une date sans information horaire. Cette transformation est essentielle pour permettre un regroupement des ventes par jour.

En supprimant la composante horaire, le script garantit que toutes les ventes réalisées au cours d'une même journée sont agrégées ensemble. Cette étape permet de produire des indicateurs journaliers cohérents et facilement interprétables.

9.6 Regroupement des ventes par jour

Une fois la date normalisée, le script regroupe les données par jour à l'aide de l'instruction `groupBy`. Cette opération constitue le cœur du traitement analytique réalisé dans la couche Gold.

Le regroupement par date permet de passer d'un niveau transactionnel, où chaque ligne représente une vente, à un niveau synthétique, où chaque ligne représente une journée d'activité. Cette transformation est indispensable pour produire des indicateurs temporels exploitables dans des tableaux de bord ou des rapports.

9.7 Calcul des agrégats analytiques

Sur chaque groupe de ventes journalières, le script calcule plusieurs agrégats à l'aide de fonctions analytiques standards. Le nombre de ventes est obtenu par un comptage du nombre de transactions. Le chiffre d'affaires est calculé par la somme des montants totaux des ventes. Le panier moyen est calculé comme la moyenne du montant des transactions.

Ces agrégations transforment des données détaillées en indicateurs synthétiques, tout en conservant une granularité journalière. Elles permettent d'analyser l'évolution de l'activité commerciale dans le temps et d'identifier des tendances ou des variations significatives.

9.8 Tri et présentation des résultats

Après le calcul des agrégats, les résultats sont triés par date afin de garantir une lecture chronologique cohérente. Cette organisation facilite l'analyse temporelle et permet une visualisation claire de l'évolution des indicateurs.

Le script affiche également un aperçu des résultats, ce qui permet de valider visuellement la cohérence des agrégations avant leur stockage définitif dans la couche Gold.

9.9 Écriture des données dans la couche Gold

Les résultats des agrégations sont ensuite écrits dans la couche Gold au format Delta Lake. Cette étape marque la finalisation du processus analytique. Les données stockées à ce niveau sont prêtes à être consommées directement par des outils de reporting ou de Business Intelligence.

L'utilisation du format Delta Lake garantit la fiabilité et la traçabilité des indicateurs produits. Elle permet également de rejouer les calculs ou de revenir à des versions antérieures si nécessaire.

9.10 Optimisation pour le reporting et la Business Intelligence

La structure des données stockées dans la couche Gold est spécifiquement adaptée aux besoins du reporting. Les indicateurs sont pré-calculés, ce qui réduit considérablement le temps de réponse des outils de visualisation et des requêtes analytiques.

Cette approche permet aux utilisateurs métiers d'accéder rapidement à des informations clés sans avoir à manipuler des volumes importants de données transactionnelles. La couche Gold constitue ainsi une interface efficace entre le pipeline de données et les outils décisionnels.

9.11 Conclusion sur la couche Gold

La couche Gold représente l'aboutissement logique du pipeline Lakehouse mis en place dans ce projet. Elle transforme des données nettoyées et structurées en indicateurs métiers directement exploitables. Grâce à cette couche, les données acquièrent une réelle valeur décisionnelle et peuvent être utilisées pour le pilotage de l'activité, l'analyse de la performance et la prise de décision stratégique.

10 Génération du rapport final

10.0.1 Rôle du script `09_generer_rapport.py`

Le script `09_generer_rapport.py` constitue la dernière étape du pipeline de données mis en place dans ce projet. Contrairement aux scripts précédents, dont l'objectif principal était essentiellement technique — ingestion, historisation, nettoyage ou agrégation — ce script est orienté vers la valorisation métier des données. Il marque ainsi le passage définitif du domaine du Data Engineering vers celui de l'aide à la décision.

À ce stade du pipeline, les données ont déjà subi l'ensemble des transformations nécessaires. Elles ont été extraites depuis la base de données PostgreSQL, stockées dans l'architecture Lakehouse, nettoyées dans la couche Silver, puis agrégées dans la couche Gold. Le rôle de ce script n'est donc plus de transformer les données, mais de les exploiter afin d'en extraire des informations synthétiques et directement interprétables par des utilisateurs finaux.

10.0.2 Lecture des données issues de la couche Gold

La première opération réalisée par le script consiste à lire les données stockées dans la couche Gold, plus précisément la table contenant les ventes agrégées par jour. Cette table représente déjà un niveau avancé de synthèse, car chaque enregistrement correspond à une journée d'activité commerciale et non plus à une transaction individuelle.

Le choix de s'appuyer exclusivement sur la couche Gold garantit que les données utilisées pour le rapport sont fiables, cohérentes et prêtes pour l'analyse. Toutes les problématiques liées à la qualité des données ont été traitées en amont, ce qui permet au script de se concentrer uniquement sur l'analyse et la restitution de l'information. Cette séparation claire entre préparation des données et exploitation analytique renforce la lisibilité et la robustesse du pipeline global.

10.0.3 Calcul des statistiques globales

Après le chargement des données agrégées, le script procède au calcul de statistiques globales couvrant l'ensemble de la période analysée. Ces statistiques incluent notamment le nombre total de ventes, le chiffre d'affaires cumulé ainsi que le panier moyen global.

Ces indicateurs offrent une vision synthétique de l'activité commerciale. Ils permettent de répondre à des questions stratégiques fondamentales telles que le volume total d'activité ou la performance financière globale du système étudié. Le calcul de ces indicateurs à partir de données déjà agrégées garantit leur cohérence et leur fiabilité, tout en limitant la complexité des traitements nécessaires.

10.0.4 Analyse des jours les plus performants

En complément des statistiques globales, le script met en évidence les journées les plus performantes en termes de chiffre d'affaires. Cette analyse permet d'identifier des pics d'activité et de comparer les performances quotidiennes entre elles.

L'identification des meilleurs jours constitue un élément clé de l'analyse décisionnelle. Elle peut révéler des tendances, des comportements saisonniers ou l'impact d'événements ponctuels sur l'activité commerciale. Cette ap-

proche démontre la capacité du pipeline à produire des insights exploitables à partir de données agrégées, sans nécessiter de traitements analytiques complexes supplémentaires.

10.0.5 Production d'un rapport lisible et orienté métier

Les résultats produits par le script sont présentés sous la forme d'un rapport textuel clair et structuré. Ce rapport est volontairement conçu pour être compréhensible par des utilisateurs non techniques, tels que des responsables ou des décideurs. Les indicateurs sont affichés de manière lisible, sans exposer les détails techniques liés au traitement des données.

Cette restitution met en évidence la finalité du pipeline : transformer des données brutes en information à forte valeur ajoutée. Le rapport démontre que l'architecture mise en place ne se limite pas à une succession d'étapes techniques, mais aboutit à une production d'information directement exploitable pour la prise de décision.

10.0.6 Finalité du pipeline de données

Le script `09_generer_rapport.py` représente l'aboutissement logique du pipeline de données développé dans ce projet. Il matérialise la transformation progressive des données, depuis leur état brut dans le système transactionnel jusqu'à leur restitution sous forme d'indicateurs synthétiques.

Grâce à cette étape finale, l'ensemble des choix techniques réalisés tout au long du projet trouve une justification concrète. Le pipeline démontre sa capacité à produire une information fiable, compréhensible et orientée métier, répondant pleinement aux objectifs d'un système décisionnel moderne.

11 Conclusion

Ce projet a permis de concevoir et de mettre en œuvre un pipeline de Data Engineering complet et cohérent, couvrant l'ensemble du cycle de vie des données, depuis leur extraction à partir d'un système transactionnel jusqu'à leur exploitation à des fins décisionnelles. L'architecture développée s'inscrit dans une démarche moderne de gestion des données, inspirée des pratiques industrielles actuelles en Data Warehouse et Big Data.

La première étape du projet a consisté à ingérer les données depuis une base de données PostgreSQL, représentant un système OLTP. Cette ingestion a été réalisée de manière contrôlée, en garantissant l'intégrité et la fidélité des données sources. Le stockage initial dans la couche Bronze du Lakehouse a permis de conserver une copie brute et traçable des données, constituant ainsi une base de référence stable pour l'ensemble des traitements ultérieurs.

La gestion de l'historique, notamment à travers la mise en œuvre du Slowly Changing Dimension de type 2 et la modélisation Data Vault, a permis de répondre à des problématiques essentielles de traçabilité et d'évolution des données. Ces mécanismes assurent la conservation des différentes versions des informations métiers, ce qui est indispensable pour réaliser des analyses temporelles fiables et pour comprendre l'évolution des entités dans le temps.

Le nettoyage et la standardisation des données dans la couche Silver ont constitué une étape clé pour améliorer la qualité globale des données. En appliquant des règles de suppression des doublons, de normalisation des formats et de validation des valeurs, cette couche a permis de transformer des données brutes en données cohérentes et exploitables. Cette étape intermédiaire garantit que les analyses reposent sur des données fiables, tout en conservant un niveau de détail suffisant.

La couche Gold a ensuite permis de produire des données analytiques à forte valeur ajoutée, sous forme d'agrégations et d'indicateurs métiers. Les données ont été structurées de manière à répondre directement aux besoins décisionnels, facilitant ainsi leur exploitation par des outils de reporting ou de Business Intelligence. Cette couche représente l'aboutissement analytique du pipeline, en transformant les données nettoyées en information synthétique et interprétable.

L'utilisation conjointe de PySpark et de Delta Lake a apporté des garanties techniques essentielles en termes de performance, de fiabilité et de traçabilité. PySpark a permis de traiter efficacement les données, tandis que Delta Lake a assuré la gestion des transactions, le versioning des données et la possibilité de rejouer les traitements. Ces choix technologiques rendent le pipeline robuste, maintenable et évolutif.

Enfin, l'architecture en couches Bronze, Silver et Gold a permis une séparation claire des responsabilités entre les différentes étapes du traitement. Cette organisation facilite la maintenance du système, améliore la lisibilité du pipeline et permet d'intégrer de nouvelles sources de données ou de nouvelles règles métier sans remettre en cause l'existant.

En conclusion, ce projet illustre de manière concrète les bonnes pratiques industrielles en matière de Data Engineering et de conception de systèmes décisionnels modernes. Il démontre la capacité à concevoir une architecture complète, structurée et orientée valeur métier, répondant aux exigences actuelles des projets de Data Warehouse et de Big Data.

12 Captures d'écran

```
(venv) (base) nadaeazzouhour@NADAC TP_DataWarehouse % nano test_installation.py

(venv) (base) nadaeazzouhour@NADAC TP_DataWarehouse % python test_installation.py

PySpark version: 4.1.0
Delta Lake est installé ✓
Pyscopg2 version: 2.9.11 (dt dec pq3 ext lo64)
Pandas version: 2.3.3

Toutes les bibliothèques sont installées avec succès !
(venv) (base) nadaeazzouhour@NADAC TP_DataWarehouse %
```

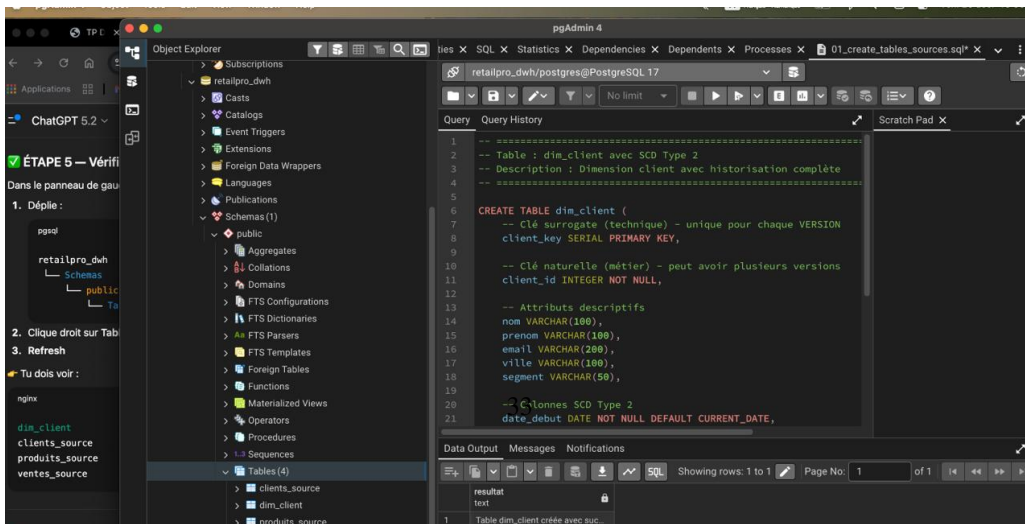
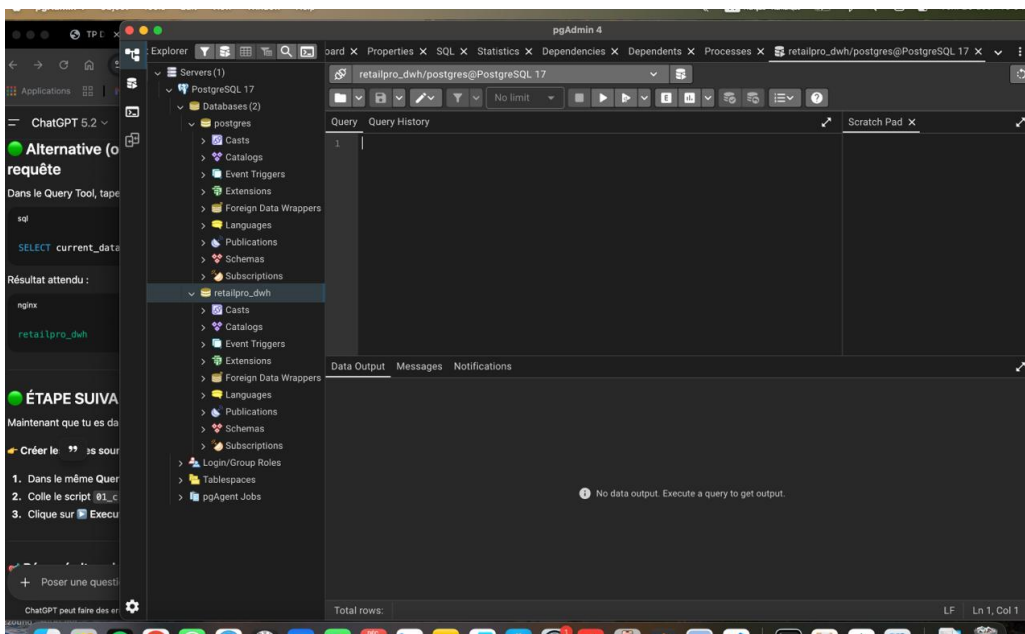
```
Toutes les bibliothèques sont installées avec succès !
(venv) (base) nadaeazzouhour@NADAC TP_DataWarehouse % cd ~/TP_DataWarehouse

(venv) (base) nadaeazzouhour@NADAC TP_DataWarehouse % mkdir drivers

(venv) (base) nadaeazzouhour@NADAC TP_DataWarehouse % mv /Users/nadaeazzouhour/Downloads/postgresql-42.7.7.jar ~/TP_DataWarehouse/drivers/

(venv) (base) nadaeazzouhour@NADAC TP_DataWarehouse % ls ~/TP_DataWarehouse/drivers

postgresql-42.7.7.jar
(venv) (base) nadaeazzouhour@NADAC TP_DataWarehouse %
```



```
TP_DataWarehouse — -zsh — 102x34

Requirement already satisfied: psycopg2-binary in ./venv/lib/python3.11/site-packages (2.9.11)
(venv) (base) nadaezzouhour@NADAC TP_DataWarehouse % nano 03_load_scd_type2.py
(venv) (base) nadaezzouhour@NADAC TP_DataWarehouse % python 03_load_scd_type2.py

Bibliothèques importées
Configuration : connexion à retailpro_dwh sur localhost
Connexion à PostgreSQL réussie
3 clients lus depuis la source

=====
DÉBUT DU PROCESSUS SCD TYPE 2
=====

Traitement du client 1...
✓ Nouveau client inséré : client_id=1

Traitement du client 2...
✓ Nouveau client inséré : client_id=2

Traitement du client 3...
✓ Nouveau client inséré : client_id=3

=====
RÉSUMÉ DU CHARGEMENT
=====
Nouveaux clients insérés : 3
Nouvelles versions créées : 0
Clients inchangés : 0
Total traité : 3
=====

✓ Script SCD Type 2 terminé avec succès
(venv) (base) nadaezzouhour@NADAC TP_DataWarehouse %
```

pgAdmin 4

retailpro_dwh/postgres@PostgreSQL 17

Query

```
1 SELECT client_id, nom, ville, segment, version, est_courant
2 FROM dim_client
3 ORDER BY client_id, version;
```

Data Output

client_id	nom	ville	segment	version	est_courant
integer	character varying (100)	character varying (100)	character varying (50)	integer	boolean
1	Dupont	Paris	Gold	1	true
2	Martin	Lyon	Silver	1	true
3	Bernard	Marseille	Bronze	1	true

Total rows: 3 Query complete 00:00:07.712 LF Ln 3, Col 29

pgAdmin 4

retailpro_dwh/postgres@PostgreSQL 17

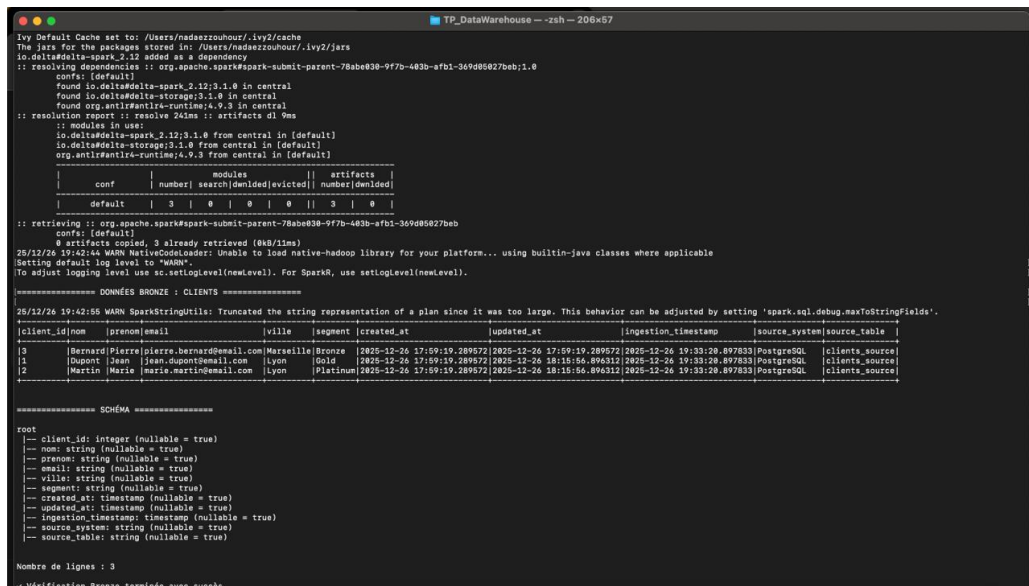
Query

```
1 -- Modifier la ville du client 1
2 UPDATE clients_source
3 SET ville = 'Lyon',
4     updated_at = CURRENT_TIMESTAMP
5 WHERE client_id = 1;
6
7 -- Modifier le segment du client 2
8 UPDATE clients_source
9 SET segment = 'Platinum',
10    updated_at = CURRENT_TIMESTAMP
11 WHERE client_id = 2;
12
13 -- Vérifier les données sources
14 SELECT client_id, nom, ville, segment
15 FROM clients_source
16 ORDER BY client_id;
17
```

Data Output

client_id	nom	ville	segment
[PK] integer	character varying (100)	character varying (100)	character varying (50)
1	Dupont	Lyon	Gold

34



```
TP_DataWarehouse -- -zsh -- 145x31

confs: [default]
found io.delta#delta-spark_2.12;3.1.0 in central
found io.delta#delta-storage;3.1.0 in central
found org.apache.spark#spark-submit-parent-80473b1b-0e20-4c2e-89f8-92ffb77622f6 in central
:: resolution report :: resolve 228ms :: artifacts dl 31ms
:: modules in use:
io.delta#delta-spark_2.12;3.1.0 from central in [default]
io.delta#delta-storage;3.1.0 from central in [default]
org.apache.spark#spark-submit-parent-80473b1b-0e20-4c2e-89f8-92ffb77622f6 from central in [default]
-----
| conf | number | modules | artifacts | | | |
|---|---|---|---|---|---|---|
| default | 3 | 0 | 0 | 0 | 3 | 0 |
-----

:: retrieving :: org.apache.spark#spark-submit-parent-80473b1b-0e20-4c2e-89f8-92ffb77622f6
0 artifacts copied, 3 already retrieved (0kB/10ms)
25/12/26 19:47:02 WARN NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
=====
TRANSFORMATION SILVER : BRONZE -> SILVER
=====
25/12/26 19:47:12 WARN SparkStringUtils: Truncated the string representation of a plan since it was too large. This behavior can be adjusted by setting 'spark.sql.debug.maxToStringFields'.
Clients Bronze : 3
Clients Silver (après nettoyage) : 3
✓ Silver clients créé avec succès
✓ Transformation Silver terminée
(venv) (base) nadaezzouhour@NADAC TP_DataWarehouse %
```

```
TP_DataWarehouse -- -zsh -- 145x31

io.delta#delta-storage;3.1.0 from central in [default]
org.apache.spark#spark-submit-parent-314c3f9f-c3fc-45af-afid-2895a1bb33f5 from central in [default]
-----
| conf | number | modules | artifacts | | | |
|---|---|---|---|---|---|---|
| default | 3 | 0 | 0 | 0 | 3 | 0 |
-----

:: retrieving :: org.apache.spark#spark-submit-parent-314c3f9f-c3fc-45af-afid-2895a1bb33f5
confs: [default]
0 artifacts copied, 3 already retrieved (0kB/10ms)
25/12/26 19:53:03 WARN NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
=====
CRÉATION DE LA COUCHE GOLD
=====
APERÇU DES AGRÉGATIONS GOLD :
25/12/26 19:53:12 WARN SparkStringUtils: Truncated the string representation of a plan since it was too large. This behavior can be adjusted by setting 'spark.sql.debug.maxToStringFields'.
-----
|date|nb_ventes|ca_total|panier_moyen|
-----
|2025-12-26|3|1109.96|369.986667|
-----
✓ Gold ventes quotidiennes créé avec succès
✓ Agrégation Gold terminée
(venv) (base) nadaezzouhour@NADAC TP_DataWarehouse %
```

```
TP_DataWarehouse -- -zsh -- 145x40

io.delta#delta-spark_2.12;3.1.0 from central in [default]
io.delta#delta-storage;3.1.0 from central in [default]
org.apache.spark#spark-submit-parent-ab9d5c98-3c80-4e14-8fce-30f1141cd6aa from central in [default]
-----
| conf | number | modules | artifacts | | | |
|---|---|---|---|---|---|---|
| default | 3 | 0 | 0 | 0 | 3 | 0 |
-----

:: retrieving :: org.apache.spark#spark-submit-parent-ab9d5c98-3c80-4e14-8fce-30f1141cd6aa
confs: [default]
0 artifacts copied, 3 already retrieved (0kB/8ms)
25/12/26 19:55:16 WARN NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
=====
RAPPORT DATA WAREHOUSE - 2025-12-26 19:55
=====
25/12/26 19:55:25 WARN SparkStringUtils: Truncated the string representation of a plan since it was too large. This behavior can be adjusted by setting 'spark.sql.debug.maxToStringFields'.
STATISTIQUES GLOBALES
-----
Total des ventes : 3
Chiffre d'affaires : 1109.96 €
Panier moyen global : 369.99 €
-----
TOP 5 MEILLEURS JOURS (PAR CA)
-----
-----
|date|nb_ventes|ca_total|panier_moyen|
-----
|2025-12-26|3|1109.96|369.986667|
-----
✓ Rapport généré avec succès
(venv) (base) nadaezzouhour@NADAC TP_DataWarehouse %
```